

Modulo 0B — Pipe, Redirect e Filtri

Corso: Lab Amministrazione di Sistemi T

Prerequisito: Modulo 0A completato (filesystem, permessi)

VM: Vagrant + Debian 12 (`cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`)

Il concetto fondamentale: flussi di dati

Su Linux ogni processo lavora con tre flussi di dati:

Flusso	Numero	Descrizione
stdin	0	Input standard — da tastiera per default
stdout	1	Output standard — verso il terminale per default
stderr	2	Output degli errori — verso il terminale per default

Redirect e pipe servono a **ridirigere questi flussi**: invece di leggere da tastiera puoi leggere da file, invece di stampare a schermo puoi scrivere su file, invece di passare l'output all'utente puoi passarlo a un altro comando.

Parte 1 — Redirect

1.1 Creare materiale di lavoro

Prima di tutto, crea i file che useremo in questa lezione:

```
mkdir ~/lab0B && cd ~/lab0B
```

```
cat > frutta.txt << 'EOF'
mela
banana
pera
mela
arancia
banana
mela
kiwi
pera
EOF
```

Verifica il contenuto:

```
cat frutta.txt
```

Output atteso: le 9 righe con i nomi dei frutti, nell'ordine in cui le hai scritte.

1.2 Redirect output: >

L'operatore > prende lo stdout di un comando e lo scrive in un file, **sovrascrivendo** il contenuto esistente.

```
echo "primo contenuto" > output.txt
cat output.txt
```

Output atteso: primo contenuto

```
echo "secondo contenuto" > output.txt
cat output.txt
```

Output atteso: secondo contenuto — il primo è sparito. > sovrascrive sempre.

1.3 Redirect in append: >>

L'operatore >> aggiunge in coda senza sovrascrivere.

```
echo "riga uno" > log.txt
echo "riga due" >> log.txt
echo "riga tre" >> log.txt
cat log.txt
```

Output atteso:

```
riga uno
riga due
riga tre
```

Questo è il comportamento usato dai sistemi di logging: ogni evento viene aggiunto in coda al file di log senza cancellare la storia precedente.

1.4 Redirect input: <

L'operatore < fornisce il contenuto di un file come stdin a un comando.

```
wc -l < frutta.txt
```

Output atteso: 9 — il numero di righe nel file.

wc (word count) con il flag -l conta le righe. Senza redirect avresti dovuto passargli il nome del file come argomento (wc -l frutta.txt); con < gli passi lo stesso contenuto tramite stdin. In questo caso il risultato è identico, ma la distinzione diventa importante quando lavori con output di altri comandi.

1.5 Redirect degli errori: 2>

`stderr` ha numero 2. Se vuoi catturare i messaggi di errore in un file:

```
cat file_che_non_esiste.txt 2> errori.txt
cat errori.txt
```

Output atteso nel file: `cat: file_che_non_esiste.txt: No such file or directory`

Per ridirigere sia `stdout` che `stderr` nello stesso file:

```
cat frutta.txt file_inesistente.txt > tutto.txt 2>&1
cat tutto.txt
```

`2>&1` significa “manda `stderr` dove sta andando `stdout`”. Vedrai nel file sia il contenuto di `frutta.txt` che il messaggio di errore per il file inesistente.

Parte 2 — Pipe

2.1 Il concetto

Il carattere `|` connette lo **stdout di un comando** allo **stdin del successivo**. Invece di scrivere output intermedi su file, i dati fluiscono direttamente da un programma all’altro in memoria.

```
comando1 | comando2 | comando3
```

L’output di `comando1` entra in `comando2`, il cui output entra in `comando3`.

2.2 Prime pipe

```
cat frutta.txt | wc -l
```

Output atteso: 9

Equivale a `wc -l frutta.txt`, ma ora capisci il meccanismo: `cat` manda il contenuto su `stdout`, `wc -l` lo riceve su `stdin` e conta le righe.

```
cat frutta.txt | sort
```

Output atteso: i frutti in ordine alfabetico.

```
cat frutta.txt | sort | uniq
```

Output atteso: i frutti in ordine alfabetico, **senza duplicati**. `uniq` rimuove le righe consecutive identiche — per questo funziona correttamente solo dopo `sort`.

2.3 Contare elementi unici

```
cat frutta.txt | sort | uniq | wc -l
```

Output atteso: 6 — ci sono 6 frutti distinti nella lista.

Questa è una pipeline tipica: ordina → rimuovi duplicati → conta. La userai spesso per analizzare log e output di tool.

2.4 `uniq -c` — conta le occorrenze

```
cat frutta.txt | sort | uniq -c
```

Output atteso (l'ordine può variare leggermente):

```
1 arancia
2 banana
1 kiwi
3 mela
2 pera
```

Il numero a sinistra indica quante volte ogni elemento appare. Utile per trovare gli indirizzi IP che compaiono più spesso in un log di accessi, ad esempio.

2.5 `head` e `tail` — prime e ultime righe

```
cat frutta.txt | head -3
cat frutta.txt | tail -3
```

Output atteso: le prime 3 e le ultime 3 righe del file rispettivamente.

```
tail -f /var/log/syslog
```

Il flag `-f` (follow) tiene il file aperto e mostra le nuove righe in tempo reale — fondamentale per monitorare log di sistema mentre accade qualcosa. Premi `Ctrl+C` per uscire.

Parte 3 — `grep`

`grep` è lo strumento di ricerca per testo. Cerca righe che corrispondono a un pattern e le stampa su stdout.

3.1 Ricerca base

```
grep "mela" frutta.txt
```

Output atteso: tutte le righe che contengono la parola “mela”.

```
grep -c "mela" frutta.txt
```

Output atteso: 3 — il flag `-c` conta le corrispondenze invece di stamparle.

```
grep -i "MELA" frutta.txt
```

Output atteso: stesse righe di prima — il flag `-i` rende la ricerca case-insensitive.

3.2 grep su output di altri comandi

```
ls /etc | grep "cron"
```

Output atteso: tutte le voci di `/etc` che contengono “cron” nel nome (`cron.d`, `cron.daily`, `crontab`, ecc.).

```
cat /etc/passwd | grep "vagrant"
```

Output atteso: la riga del file `/etc/passwd` relativa all’utente `vagrant`. `/etc/passwd` contiene la lista di tutti gli utenti del sistema.

3.3 grep con flag utili

```
grep -n "mela" frutta.txt
```

Output atteso: le righe con “mela” precedute dal numero di riga. `-n` è utile quando lavori su file grandi e devi localizzare un match.

```
grep -v "mela" frutta.txt
```

Output atteso: tutte le righe che **non** contengono “mela”. `-v` inverte il filtro — fondamentale per escludere rumore dai log.

Parte 4 — Pipeline avanzate

4.1 Analisi del file passwd

```
cat /etc/passwd | cut -d: -f1 | sort
```

`cut` estrae colonne da testo strutturato: `-d:` usa il carattere `:` come delimitatore, `-f1` prende il primo campo. In `/etc/passwd` il primo campo è il nome utente. Il risultato è la lista alfabetica di tutti gli utenti del sistema.

4.2 Trova i processi in esecuzione

```
ps aux | grep "bash"
```

`ps aux` mostra tutti i processi in esecuzione. Passato a `grep "bash"` filtra solo quelli che contengono la parola `bash`. Questa pipeline è usata continuamente per verificare se un servizio è attivo.

4.3 Conta i file per tipo in /etc

```
ls /etc | grep "\.conf$" | wc -l
```

`\.conf$` è un’espressione regolare: `\.` cerca un punto letterale, `conf` il testo, `$` indica fine riga. Il risultato è il numero di file `.conf` in `/etc`.

Esercizio finale

Senza guardare le soluzioni, costruisci le pipeline che rispondono a queste domande usando i comandi della lezione:

1. Quante righe contiene il file `/etc/passwd`?
2. Quanti utenti distinti sono nel file `/etc/passwd` (primo campo, separatore `:`)?
3. Quali righe di `/var/log/syslog` contengono la parola “error”? (maiuscolo o minuscolo)
4. Mostra solo le ultime 5 righe di `/var/log/auth.log`

Riepilogo operatori e comandi

Comando/Operatore	Funzione
<code>></code>	Redirect stdout su file (sovrascrive)
<code>>></code>	Redirect stdout su file (aggiunge in coda)
<code><</code>	Fornisce file come stdin
<code>2></code>	Redirect stderr su file
<code>2>&1</code>	Redirect stderr su stdout
<code>\ </code>	Pipe: stdout di sinistra → stdin di destra
<code>grep pattern</code>	Filtra righe che contengono il pattern
<code>grep -v</code>	Filtra righe che NON contengono il pattern
<code>grep -i</code>	Case-insensitive
<code>grep -n</code>	Mostra numeri di riga
<code>grep -c</code>	Conta le corrispondenze
<code>sort</code>	Ordina righe alfabeticamente
<code>uniq</code>	Rimuove righe duplicate consecutive
<code>uniq -c</code>	Conta occorrenze di ogni riga unica
<code>wc -l</code>	Conta le righe
<code>head -N</code>	Prime N righe
<code>tail -N</code>	Ultime N righe
<code>tail -f</code>	Segue il file in tempo reale
<code>cut -d: -f1</code>	Estrae il primo campo usando <code>:</code> come delimitatore

Connessione con Security

Tutto quello che hai imparato qui è la base operativa dei lab di Sicurezza: `- nmap -sV target | grep "open" — filtra le porte aperte dall'output di Nmap - cat auth.log | grep "Failed password" | wc -l — conti i tentativi di login falliti - cat access.log | cut -d" " -f1 | sort | uniq -c | sort -rn | head -10 — trovi i 10 IP che hanno fatto più richieste al server web`

Il pattern è sempre lo stesso: produci output grezzo → filtra → analizza.

Collegati

- `[[appunti_modulo0B_pipe_redirect_filtri]]`

Hub: [[master_map_studio]] · [[glossario_sysadm]] · [[concept_maps]] · [[troubleshooting_vm]]
· [[metodo_studio_esami_pratici]]